0-LOOP

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Proyecto: Sistema de Punto de Venta (POS) B2B - Farmacia POS-AWOS

Materia: Aplicaciones Web Orientadas a Servicios (AWOS)

Docente: Dr.Jazmín Rodríguez Flores

Integrantes (Equipo de Alto Rendimiento):

Angel Daniel Meneses Castilleja: Líder Frontend, Arquitectura UI/UX, BLoC y Generación Documental.

Samuel de Jesús Alavez Lugo: Líder Backend Transaccional, y NoSQL (Firestore).

Karel Galindo Domingo: Líder Backend Inventario, Motor Relacional (PostgreSQL) y Algoritmos ACID.

Zempoala, Hidalgo 1 de marzo de 2026

0-LOOP

ÍNDICE

1. Introducción y Objetivo
1.1 Justificación del Proyecto
1.2 Alcance y Límites del Sistema

2. Normativas y Estándares de Calidad
2.1 ISO/IEC 25010 (Calidad del Producto de Software)
2.2 IEEE 830-1998 (Especificación de Requisitos)
2.3 IEEE 829-2008 (Documentación de Pruebas)

3. Metodología de Desarrollo y Herramientas
3.1 Stack Tecnológico (Cliente, Orquestador y Motor Relacional)
3.2 Defensa y Justificación de la Metodología Ágil (Scrum + GitFlow) *[Cubre Puntos 1 y 5 de la rúbrica]*
3.3 Herramientas de Gestión y Trazabilidad (Trello y GitHub) *[Cubre Puntos 2 y 12 de la rúbrica]*

4. Arquitectura de Software y Patrones de Diseño
4.1 Arquitectura Orientada a Servicios (SOA) y Microservicios
4.1.1 Diagrama de Arquitectura General del Sistema
4.2 Diagrama de Paquetes y Clases del Cliente (Flutter) *[Cubre Punto 3 de la rúbrica]*
4.3 Implementación de Patrones y Principios de Diseño en el Cliente *[Cubre Puntos 6 y 7 de la rúbrica]*
4.4 Diagrama de Paquetes y Clases del Servidor (Node.js & FastAPI) *[Cubre Punto 4 de la rúbrica]*
4.5 Implementación de Patrones y Principios de Diseño en el Servidor *[Cubre Puntos 6 y 7 de la rúbrica]*

5. Arquitectura y Diseño de Bases de Datos Híbridas
5.1 Justificación del Modelo Híbrido (Relacional + Documental)
5.2 Base de Datos Relacional: Inventario y Catálogo (PostgreSQL) *[Cubre Punto 5 de la rúbrica]*
5.3 Diagrama Entidad-Relación (ERD)
5.4 Base de Datos No Relacional: Transacciones y Roles (Firestore) *[Cubre Punto 5 de la rúbrica]*
5.5 Reglas de Integridad y Restricciones de Dominio (Constraints)

6. Diseño de APIs REST (Contratos de Integración)
6.1 Endpoints Transaccionales (Node.js)
6.2 Endpoints de Inventario y FEFO (FastAPI)
6.3 Estructura de Requests y Responses (Payloads JSON)

7. Especificación de Requerimientos de Software (Estándar IEEE 830)
7.1 Perspectiva y Funciones del Producto
7.2 Requerimientos Funcionales Estrictos

7.3 Requerimientos No Funcionales (RNF) - ISO/IEC 25010

8. Product Backlog: Historias de Usuario y Casos de Uso
8.1 Listado de Historias de Usuario y Estimaciones (Scrum) *[Cubre Punto 2 de la rúbrica]*
8.2 Diagrama General de Casos de Uso (UML)
8.3 Especificación Formal de Casos de Uso (Core Business)

9. Estrategia Integral de Pruebas (Estándar IEEE 829)
9.1 Plan de Pruebas (Test Plan)
9.2 Diseño y Especificación de Pruebas
9.2.1 Documentación de Pruebas del Cliente (Mínimo 3 documentadas y explicadas) *[Cubre Puntos 6 y 11 de la rúbrica]*
9.2.2 Documentación de Pruebas del Servidor (Mínimo 3 documentadas y explicadas) *[Cubre Puntos 7 y 11 de la rúbrica]*
9.3 Reporte de Incidentes y Resumen de Ejecución

10. Métricas de Control Ágil y Definition of Done (DoD)
10.1 Métricas de Rendimiento (Velocity y SLAs)
10.2 Criterios de Aceptación y DoD Exhaustivo

11. Despliegue, Reportes y Demostración Funcional
11.1 Arquitectura de Despliegue en 2 Sitios Diferentes (Firebase Hosting + Render) *[Cubre Punto 13 de la rúbrica]*
11.2 Módulo de Generación de Reportes del Sistema *[Cubre Punto 15 de la rúbrica]*
11.3 Evidencia de Funcionalidad: Videos Explicativos (Cliente y Servidor) *[Cubre Puntos 8 y 9 de la rúbrica]*

12. Autoevaluación del Equipo de Desarrollo *[Cubre Punto 16 de la rúbrica]*

13. Enlaces y Repositorios Oficiales del Proyecto

14. Bibliografía y Referencias Técnicas

1. Introducción y Objetivo

El Sistema de Punto de Venta (POS) "Farmacia AWOS" es una plataforma orientada a la gestión farmacéutica y operación de mostrador. A diferencia de un POS comercial genérico, este sistema resuelve necesidades críticas del sector de la salud: la trazabilidad de lotes médicos, la prevención de venta de medicamentos caducados y la retención de recetas para sustancias controladas.

Contexto y Entorno de Operación:

- **Tipo de sistema: Sistema de información transaccional (OLTP - *Online Transaction Processing*) para operaciones de punto de venta en tiempo real.**
- **Usuarios principales: Cajeros (operación de mostrador), Gerentes (supervisión, cortes de caja y auditoría) y Almacenistas (gestión de inventario físico y mermas).**
- **Entorno operativo: El sistema está diseñado para ejecutarse en entornos de retail que pueden presentar conexión a internet intermitente, por lo que la interfaz cliente implementa mecanismos de resiliencia ante fallos de red.**

El objetivo general es desarrollar un sistema híbrido (Web y Móvil) bajo una Arquitectura Orientada a Servicios (SOA), garantizando la integridad del inventario físico mediante el algoritmo de despacho FEFO (First Expires, First Out) y asegurando el cumplimiento de las normativas operativas de salud pública.

1.1 Justificación del Proyecto

El manejo ineficiente del inventario en una farmacia representa una pérdida económica directa por caducidad de productos y un riesgo de cumplimiento normativo. Este proyecto se justifica por la necesidad de implementar una herramienta que automatice estas reglas de negocio desde el momento del cobro.

El uso del algoritmo FEFO reduce drásticamente el riesgo de pérdidas económicas causadas por medicamentos caducados, priorizando la rotación de inventario con fechas de expiración más próximas. Adicionalmente, el sistema mejora la trazabilidad sanitaria, permitiendo

identificar los lotes exactos utilizados en cada venta, lo cual facilita auditorías regulatorias. Asimismo, prevendrá omisiones legales al bloquear algorítmicamente la venta de antibióticos si el operador no registra previamente la Cédula Profesional del médico prescriptor (regla de negocio simulada para el alcance del proyecto).

1.2 Alcance y Límites del Sistema

Para garantizar la viabilidad del desarrollo y la estabilidad de la arquitectura, se definen explícitamente los siguientes límites funcionales:

El sistema SÍ incluye (Alcance):

- Procesamiento de ventas en mostrador mediante una interfaz de usuario responsiva.
- Gestión transaccional de inventario con prevención de stock negativo mediante restricciones a nivel de motor de base de datos (CHECK (stock_actual >= 0)).
- Auditoría de medicamentos controlados mediante captura de datos médicos.
- Generación de comprobantes de venta digitales (Tickets en formato PDF 80mm).
- Autenticación Single Sign-On (SSO) vía Google y control de acceso basado en roles.

El sistema NO incluye (Límites del Software):

- Integración con pasarelas de pago bancarias (Stripe, Terminales POS físicas); los pagos con tarjeta se registran únicamente de manera lógica.
- Timbrado fiscal o facturación electrónica (Conexión con el SAT).
- Conexión con APIs externas de proveedores para reabastecimiento automático.

1.3 Glosario de Términos Técnicos

Para facilitar la lectura del presente documento arquitectónico, se definen los siguientes conceptos clave:

- FEFO (First Expires, First Out): Algoritmo de gestión de inventario que prioriza el despacho de los productos con la fecha de caducidad más próxima.
- RBAC (Role-Based Access Control): Modelo de seguridad que restringe el acceso al sistema basándose en el rol del usuario (Ej. Cajero vs. Gerente).
- Patrón Saga: Patrón de arquitectura distribuida que gestiona transacciones entre múltiples servicios mediante una secuencia de

operaciones locales y mecanismos de compensación (*Rollback*) en caso de fallo.
- SSO (Single Sign-On): Mecanismo de autenticación que permite a un usuario acceder a múltiples aplicaciones con un solo inicio de sesión.
- Debounce: Patrón de programación en el Frontend que retrasa la ejecución de una función (como una petición HTTP) hasta que el usuario deja de interactuar por un tiempo determinado, optimizando el ancho de banda.
- ACID: Propiedades de las transacciones en bases de datos relacionales (Atomicidad, Consistencia, Aislamiento, Durabilidad) que garantizan la integridad de los datos.

---

2. Normativas y Estándares de Calidad

El ciclo de vida de este software se rige bajo los siguientes estándares internacionales, aplicados directamente en la arquitectura del proyecto:

2.1 ISO/IEC 25010 (Calidad del Producto de Software)

- Rendimiento (Eficiencia): El sistema garantiza una latencia de respuesta inferior a 500 ms en búsquedas predictivas en el catálogo mediante la implementación de un patrón *Debounce* en el cliente, evitando consultas excesivas al servidor.
- Seguridad: Los servicios están protegidos mediante autenticación basada en JSON Web Tokens (JWT) y validación de firmas criptográficas.
- Fiabilidad: La interfaz de usuario implementa mecanismos de recuperación de estado, interceptando caídas de red para permitir reintentos manuales sin perder los datos del carrito de compras.
- Mantenibilidad: La arquitectura basada en servicios desacoplados permite actualizar componentes individuales (por ejemplo, el servicio de inventario en Python) sin afectar la totalidad del sistema ni requerir caídas globales de servicio.

2.2 IEEE 830-1998 (Especificación de Requisitos)

Se utilizó este estándar (Sección 7) para definir la estructura de la Especificación de Requisitos de Software (SRS), separando rigurosamente los requerimientos funcionales (comportamiento del sistema) de los requerimientos no funcionales (restricciones de latencia, disponibilidad y arquitectura).

2.3 IEEE 829-2008 (Documentación de Pruebas)

La estrategia de validación (Sección 9) adopta los lineamientos de este estándar para estructurar el Plan de Pruebas, el Diseño de Casos de Prueba (clasificados en Unidad, Integración y E2E) y el Reporte de Incidentes.

---

3. Metodología de Desarrollo y Herramientas

3.1 Stack Tecnológico

El proyecto implementa un ecosistema de tecnologías seleccionadas por su idoneidad para cada capa de la arquitectura SOA:

- Cliente / Frontend: Flutter 3.19 (Dart).
- API Gateway / Orquestador: Node.js 20 + Express.
- Servicio de Dominio (Inventario): Python 3.11 + FastAPI.
- Bases de Datos Híbridas: PostgreSQL 18 (Relacional) y Firebase Firestore (Documental).

3.2 Defensa y Justificación de la Metodología Ágil (Scrum + GitFlow)

Para el desarrollo de la Farmacia AWOS, se seleccionó el marco de trabajo Scrum. Esta decisión se fundamenta en la naturaleza distribuida de la Arquitectura Orientada a Servicios (SOA). Scrum permite realizar *Sprints* donde se entrega funcionalidad vertical completa, mitigando el riesgo de problemas de integración tardía.

Roles dentro del equipo Scrum:

- Product Owner: Responsable de priorizar el *Product Backlog* y validar que las funcionalidades entregadas cumplan con las reglas de negocio farmacéuticas.
- Scrum Master: Encargado de facilitar el proceso de desarrollo, liderar las ceremonias y eliminar impedimentos técnicos u operativos durante los *Sprints*.
- Equipo de Desarrollo: Responsable de diseñar, codificar, probar y desplegar los incrementos del sistema en las diferentes capas (Frontend, Orquestador, DB).

Para el control de código, se implementó GitFlow. Esto asegura que la rama main contenga exclusivamente código estable y desplegable. El desarrollo de las interfaces y APIs se realiza en ramas feature/ independientes, integrándose al código base a través de *Pull Requests*.

3.3 Herramientas de Gestión y Trazabilidad

- **Trello (Tablero Kanban): Administra el *Product Backlog* y da seguimiento a las Historias de Usuario (HU), permitiendo evaluar el ciclo de tiempo de desarrollo (*Cycle Time*).**
- **GitHub: Consolida el control de versiones en un repositorio unificado, proveyendo evidencia del historial de *commits* de cada integrante del equipo.**

3.4 Cronograma de Sprints (Release Plan)

El desarrollo se estructuró en 4 iteraciones (*Sprints*) de 2 semanas cada uno, priorizando el valor de negocio de mayor a menor riesgo arquitectónico:

- **Sprint 1 (Fundamentos y Seguridad): Configuración del Monorepositorio, Autenticación SSO (Google) y Middleware RBAC.**
- **Sprint 2 (Catálogo e Inventario Core): Modelado de PostgreSQL, CRUD de medicamentos, ingreso de Lotes y desarrollo del Algoritmo FEFO en FastAPI.**
- **Sprint 3 (Mostrador y Orquestación): Desarrollo de la UI del POS en Flutter (BLoC), Carrito de Compras, Búsqueda predictiva y desarrollo de la Orquestación Saga en Node.js.**
- **Sprint 4 (Auditoría, PDF y Pruebas): Generación de tickets PDF nativos, reportes de auditoría en Firestore, pruebas End-to-End (E2E) y despliegue a producción.**

---

4. Arquitectura de Software y Patrones de Diseño

4.1 Arquitectura Basada en Servicios Desacoplados (SOA)

El sistema abandona el diseño monolítico tradicional. La capa de presentación (Flutter) actúa como un cliente que delega la validación pesada a un Orquestador API Gateway (Node.js). Este orquestador administra la identidad y decide cuándo invocar al Servicio de Dominio de Inventario (FastAPI), el cual posee autoridad exclusiva sobre la base de datos relacional.

La comunicación entre los servicios distribuidos se realiza mediante peticiones HTTP REST, utilizando intercambio de datos en formato JSON. Cada servicio mantiene independencia de despliegue, permitiendo escalar componentes específicos del sistema según la carga de trabajo transaccional.

4.1.1 Diagrama de Arquitectura General del Sistema

(Inserta aquí un diagrama de bloques visualizando la comunicación: Flutter -> Node.js -> FastAPI -> PostgreSQL / Node.js -> Firestore)

4.1.2 Diagrama de Secuencia Transaccional (Orquestación Saga)

(Inserta aquí un diagrama de secuencia UML detallando el flujo de venta desde el Cajero hasta la compensación entre Node.js y FastAPI)

4.2 Diagrama de Paquetes y Clases del Cliente (Flutter)

El cliente web/móvil está estructurado rigurosamente bajo los principios de Clean Architecture, dividiendo el sistema en paquetes que aíslan las responsabilidades. El diseño sigue el principio de Inversión de Dependencias, donde la capa de presentación depende de abstracciones definidas en el dominio y no de implementaciones concretas de infraestructura:

- **Paquete presentation: Contiene las clases de interfaz de usuario (PosPage, TicketDialog, CustomTextField).**
- **Paquete domain: Centraliza la lógica de negocio y gestión de estado reactivo (clases PosBloc, AuthBloc).**
- **Paquete data: Implementa la comunicación externa (clases VentaRepository, ApiClient y DTOs).**

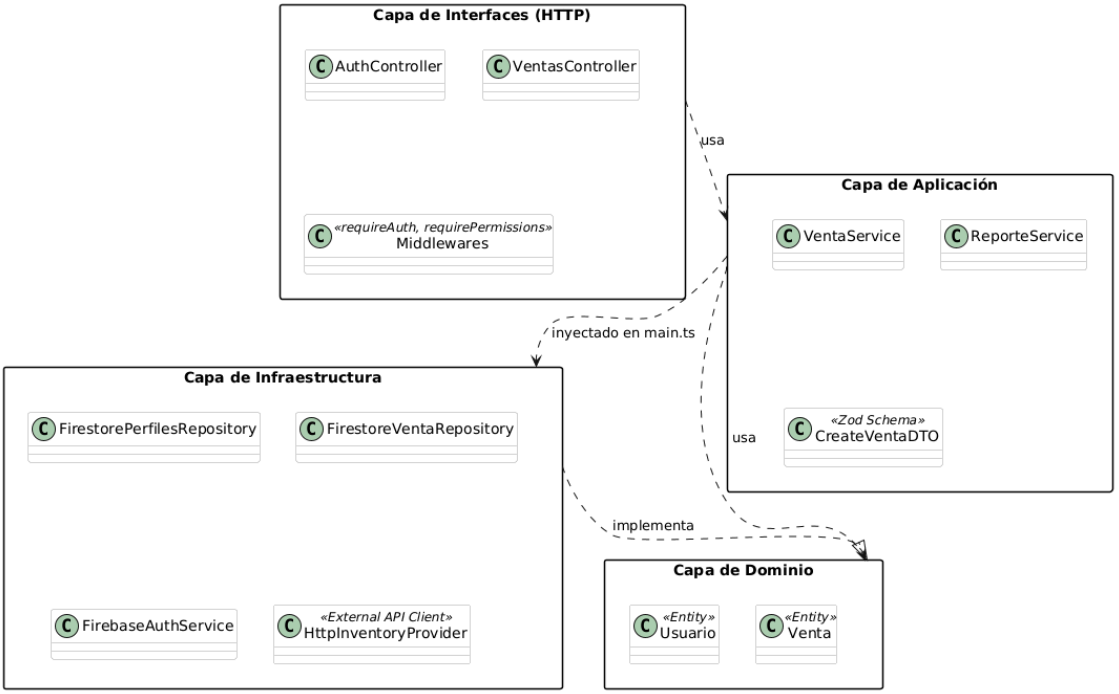
4.3 Implementación de Patrones y Principios de Diseño en el Cliente

- **Patrón BLoC (Business Logic Component): Se implementó para separar la lógica de la interfaz. La vista (PosPage) nunca altera los datos directamente; emite un evento (ej. AgregarProductoEvent). La clase PosBloc procesa la regla de negocio y emite un nuevo estado inmutable a la pantalla.**
- **Patrón Singleton: Se aplicó en la clase de configuración HTTP para asegurar que exista una única instancia del cliente de red en toda la aplicación, optimizando el consumo de memoria RAM.**
- **Repository Pattern: El acceso a datos remotos se encapsula dentro de clases repositorio (ej. VentaRepository), las cuales abstraen la comunicación HTTP con el backend, permitiendo que la lógica de negocio permanezca totalmente desacoplada de la implementación de red.**

4.4 Diagrama de Paquetes y Clases del Servidor (Node.js & FastAPI)

El API Gateway transaccional (Node.js/Express) fue desarrollado utilizando TypeScript bajo un enfoque estricto de Clean Architecture, dividiendo el sistema en 4 capas concéntricas:

- **Capa de Interfaces (HTTP): Contiene los controladores (`AuthController`, `VentasController`), definidores de rutas (`ventas.routes.ts`) y Middlewares de seguridad (`requireAuth`, `requirePermissions`, `errorHandler`).**
- **Capa de Aplicación: Define los casos de uso a través de servicios (`VentaService`, `ReporteService`) y establece los contratos de entrada/salida mediante Data Transfer Objects (DTOs) validados estrictamente con la librería `Zod` (ej. `CreateVentaDTO`).**
- **Capa de Dominio: Contiene las entidades puras del negocio (`Usuario`, `Venta`) y sus Value Objects.**
- **Capa de Infraestructura: Implementa los contratos del dominio hacia el exterior, conteniendo la persistencia (`FirestorePerfilesRepository`, `FirestoreVentaRepository`) y los clientes de red (`FirebaseAuthService`, `HttpInventoryProvider`).**
- **Diagrama:**



[ÁREA PARA KAREL: Insertar aquí un breve diagrama o descripción de las carpetas de FastAPI]

4.5 Implementación de Patrones y Principios de Diseño en el Servidor

El ecosistema del servidor implementa patrones de diseño fundamentales para garantizar el bajo acoplamiento y la integridad de las transacciones distribuidas:

Implementación en Node.js (Orquestador):

1. **Repository Pattern / DAO: Se aísla la base de datos NoSQL mediante clases como `FirestoreVentaRepository`. Si en el futuro se migra de Firestore a otro motor documental, la capa de dominio no sufrirá alteraciones.**
2. **Inyección de Dependencias (DI) Manual: En el punto de entrada del servidor (`src/main.ts`), las instancias de los repositorios y servicios externos se inyectan dinámicamente en el inicializador de la aplicación (`createApp`), facilitando la creación de *Mocks* aislados para pruebas unitarias.**
3. **Saga Pattern (Orquestación): Implementado en la clase `VentaService` trabajando en conjunto con `HttpInventoryProvider`. Este servicio dirige el flujo de venta y contiene la lógica de compensación (*Rollback*) que avisa a FastAPI si la persistencia local del ticket falla.**

Implementación en Python (FastAPI): `[ÁREA PARA KAREL: Explicar el Unit of Work implementado en Python para garantizar la integridad de las transacciones y los bloqueos SQL]`

4.6 Estrategia de Seguridad Aplicada

Para garantizar la integridad y confidencialidad de las operaciones, el sistema implementa múltiples capas de seguridad en el backend:

- **Validación Criptográfica: El SSO no delega la seguridad al cliente; Node.js verifica la firma criptográfica del token provisto por Firebase Auth antes de emitir un JWT propio.**
- **Expiración Estricta: Los tokens JWT de sesión tienen un tiempo de vida corto (1 hora), forzando la re-autenticación periódica.**
- **Sanitización de Inputs: Todas las peticiones entrantes a FastAPI son fuertemente tipadas y validadas mediante la librería Pydantic (y equivalentes en Node.js), previniendo ataques de inyección de código y asegurando que las cargas útiles (payloads) cumplan con la estructura esperada.**

---

5. Arquitectura y Diseño de Bases de Datos Híbridas

5.1 Justificación del Modelo Híbrido (Relacional + Documental)

El diseño del sistema emplea un enfoque de persistencia políglota, asignando el motor de base de datos óptimo según las necesidades del dominio:

- **Dominio Relacional (PostgreSQL): Seleccionado para el catálogo y el control de lotes físicos. La necesidad de mantener la integridad referencial y asegurar propiedades ACID (evitando vender stock inexistente mediante restricciones CHECK) hace obligatoria una base de datos SQL.**
- **Dominio Documental (Firestore): Seleccionado para almacenar los tickets de venta. Un ticket es un registro histórico e inmutable. Firestore permite almacenar documentos JSON jerárquicos que representan de forma natural la estructura de una venta (productos, cantidades, lotes, métodos de pago), reduciendo la complejidad analítica y el consumo de recursos de consultas de unión múltiple (JOINs) que serían necesarias en un modelo relacional estricto.**

5.2 Base de Datos Relacional: Inventario y Catálogo (PostgreSQL)

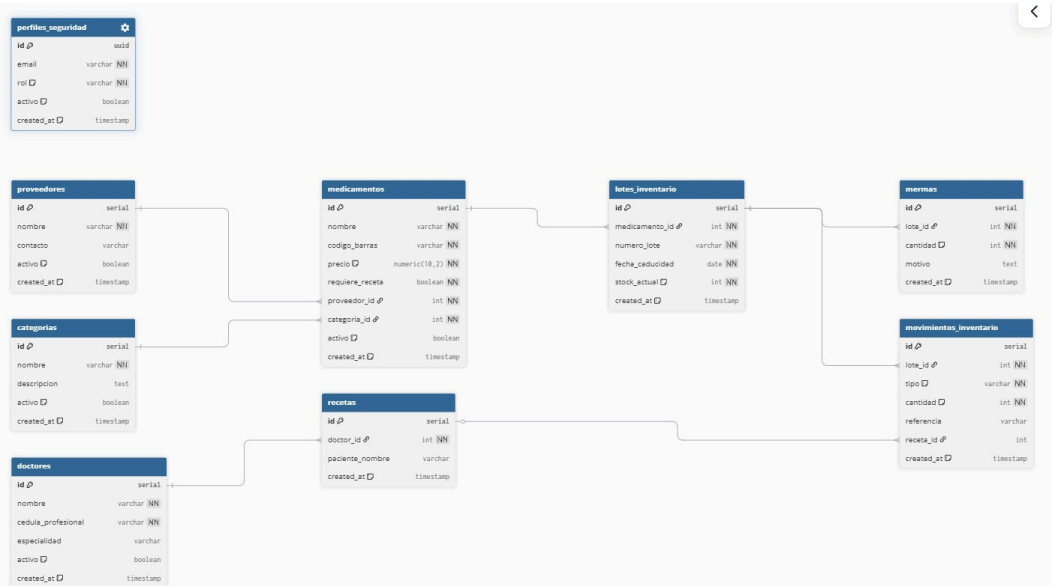
El motor relacional es la única fuente de verdad para el catálogo y las existencias físicas. Su estructura está altamente normalizada para evitar redundancias y garantizar la auditoría de cada movimiento. A continuación, se presenta el diccionario de datos de las entidades núcleo:

Tabla	Descripción y Propósito en el Dominio	Columnas Clave
medicamentos	Catálogo maestro de productos. Contiene las reglas de negocio base como la bandera de control sanitario.	id (PK), codigo_barras (Unique), precio (Numeric), requiere_receta (Boolean), proveedor_id (FK), categoria_id (FK).
lotes_inventario	Control físico de las existencias. Almacena las cajas reales ingresadas al almacén para la	id (PK), medicamento_id (FK), numero_lote (Varchar), fecha_caducidad (Date), stock_actual (Int).

	ejecución del algoritmo FEFO.	
movimientos_inventario	Libro mayor (*Ledger*) transaccional. Registra el historial inmutable de cada entrada o salida de inventario para trazabilidad.	id (PK), lote_id (FK), tipo (Varchar), cantidad (Int), referencia (Varchar).
proveedores / categorias	Tablas de catálogo auxiliar para mantener la normalización y facilitar el filtrado en las búsquedas.	id (PK), nombre (Varchar), activo (Boolean).
doctores / recetas	Tablas de control sanitario para retención de datos médicos cuando se expenden medicamentos controlados.	id (PK), cedula_profesional (Unique), paciente_nombre (Varchar).
mermas	Registro de auditoría para cajas caducadas o dañadas, justificando los	id (PK), lote_id (FK), cantidad (Int), motivo (Text).

	ajustes manuales de stock.	

5.3 Diagrama Entidad-Relación (ERD)



5.4 Base de Datos No Relacional: Transacciones y Roles (Firestore)

Para garantizar la escalabilidad y flexibilidad en el almacenamiento de documentos históricos inmutables, el Orquestador en Node.js persiste la información en Google Cloud Firestore mediante las siguientes colecciones principales, diseñadas a partir de los DTOs de la capa de aplicación:

Colección `perfiles_seguridad` (Documento de Usuario): Almacena la identidad y los permisos de acceso para el RBAC.

JSON

```
{
  "uid": "dev_user_samuel_2026",
  "email": "samuel.admin@farmacia.com",
  "nombre": "Samuel Lugo",
```

"role": "ADMIN",

"permisos": ["crear_venta", "descontar_stock", "ver_reportes", "anular_venta"]

}

Colección `ventas_tickets` (Documento de Transacción Inmutable): Almacena el registro histórico de la venta con todo su desglose, facilitando consultas rápidas para reportes sin necesidad de operaciones `JOIN` costosas.

JSON

{

"usuariold": "dev_user_samuel_2026",

"fecha": "2026-03-18T22:17:00Z",

"subtotal": 100.00,

"ivaPercentaje": 16,

"total": 116.00,

"cambio": 0,

"pagos": [

{

"tipo": "efectivo",

"monto": 116.00,

"referencia": "Venta inicial"

}

],

"lineas": [

{

"codigoProducto": "7501",

```
    "nombreProducto": "Naproxeno 500mg",
    "cantidad": 1,
    "precioUnitario": 100.00,
    "esControlado": false,
    "lote": "LOTE-TEMP"
  }
 ]
}
```


5.5 Reglas de Integridad y Restricciones de Dominio (Constraints)

Para blindar el sistema contra inconsistencias, la base de datos PostgreSQL no confía ciegamente en la validación de los microservicios, implementando barreras defensivas a nivel de motor (ACID):

1. **Prevención de Stock Negativo (`CHECK Constraint`): Se implementó la restricción `CHECK (stock_actual >= 0)` en la tabla `lotes_inventario`. Si el microservicio Node.js o FastAPI presentan una falla lógica e intentan deducir más unidades de las existentes, PostgreSQL aborta instantáneamente la transacción a bajo nivel, protegiendo la realidad física del almacén.**
2. **Bloqueo Pesimista por Concurrencia: Durante el proceso de venta, FastAPI ejecuta las consultas sobre `lotes_inventario` utilizando la cláusula `SELECT ... FOR UPDATE`. Esto bloquea los registros a nivel de fila (*Row-level locking*), impidiendo que dos cajeros deduzcan la misma caja de medicamento simultáneamente en milisegundos concurrentes (Condición de Carrera).**
3. **Integridad Referencial Estricta: Las eliminaciones en el catálogo (`medicamentos`, `proveedores`) se manejan como "bajas lógicas" (modificando la bandera booleana `activo = false`) en lugar de usar `DELETE CASCADE`, garantizando que el historial de `movimientos_inventario` y `lotes_inventario` jamás pierda sus referencias a las llaves primarias originales.**

6. Diseño de APIs REST (Contratos de Integración)

6.1 Endpoints Transaccionales (Node.js)

El API Gateway expone las siguientes rutas protegidas bajo el estándar RESTful, delegando la validación de permisos (RBAC) a Middlewares específicos (requireAuth, requirePermissions):

GET /health : Health check del estado del servidor.

POST /api/auth/login : Recepción del token de Google SSO, validación con Firebase Admin y emisión del JWT de la aplicación.

POST /api/auth/logout : Invalidación de la sesión activa.

GET /api/auth/me : Retorna el payload decodificado y los permisos del usuario activo.

POST /api/ventas/procesar : Endpoint central
GET /api/ventas/:ventaId : Consulta de un ticket inmutable específico.

GET /api/reportes/turno : Consulta agregada de tickets filtrada por usuario para el corte de caja.

6.2 Endpoints de Inventario y FEFO (FastAPI)

[ÁREA PARA KAREL: Listar POST /api/v1/inventario/descontar, POST /api/v1/inventario/compensar, GET /api/v1/medicamentos]

6.3 Estructura de Requests y Responses (Payloads JSON)

A continuación, se detallan los contratos de comunicación (Payloads) extraídos directamente de la implementación de los DTOs del API Gateway en Node.js.

1. Autenticación y Emisión de JWT (`POST /api/auth/login`)

- **Request (Body):**

JSON

{

"idToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

}

2. Orquestación de Venta - Patrón Saga (POST `/api/ventas/procesar`)

- **Request (Body desde Flutter): Representa el carrito de compras finalizado.**

JSON

{

"usuariold": "dev_user_samuel_2026",

"lineas": [

{

"codigoProducto": "7501",

"nombreProducto": "Naproxeno 500mg",

"cantidad": 1,

"precioUnitario": 100,

"esControlado": false,

"lote": "LOTE-TEMP"

}

],

"pagos": [

{

"tipo": "efectivo",

"monto": 116,

"referencia": "Venta inicial"

}

```
  ],
  "ivaPercentaje": 16,
  "subtotal": 100,
  "total": 116,
  "cambio": 0
}
```


3. Reporte de Turno y Corte de Caja (`GET /api/ventas/reporte/turno`)

- **Response (200 OK): Estructura jerárquica con metadatos para el consumo del dashboard.**

JSON


```
{
  "success": true,
  "data": {
    "ventas": [
      {
        "ventaId": "V177...",
        "total": 116,
        "estado": "procesada",
        "metodoPago": "efectivo"
      }
    ],
    "totalVentas": 116,
    "cantidadRegistros": 1,
    "subtotalPeriodo": 100,
```

```
  "ivaPeriodo": 16,
  "metadata": {
    "usuariold": "dev_user_samuel_2026",
    "fechaReporte": "2026-03-18T..."
  }
 }
}
```


---

7. Especificación de Requisitos de Software (Estándar IEEE 830)

7.1 Perspectiva y Funciones del Producto

El sistema opera bajo un modelo cliente-servidor distribuido. Su función principal es unificar la operación de caja con el control estricto de inventario médico, garantizando que ninguna venta proceda si el stock físico es insuficiente o si se omiten las regulaciones de controlados.

7.2 Requerimientos Funcionales Estrictos (RF)

Los siguientes requerimientos dictan el comportamiento obligatorio del sistema y se derivan de la lógica de negocio establecida en el Product Backlog.

Módulo 1: Identidad y Seguridad (RBAC)

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad
RF-01	Autenticación SSO	Permitir el acceso exclusivo mediante Google OAuth, emitiendo un token	HU-01, CU-01

		JWT firmado por Node.js tras validar credenciales con Firebase Admin.	
RF-02	Cierre de Sesión	Destruir el token JWT local y purgar el estado de memoria (AuthBloc) al solicitar el cierre de sesión, redirigiendo al inicio.	HU-02
RF-03	Control de Accesos (RBAC)	Bloquear la navegación a los módulos financieros y de gestión de usuarios si el payload del JWT indica el rol "Cajero".	HU-03, CU-02
RF-04	Gestión de Usuarios	Permitir al perfil "Gerente" registrar nuevos empleados, asignar roles y cambiar el estado activo/inactivo (baja lógica) en la base de datos documental.	HU-04, HU-05

Módulo 2: Catálogo Master y Banderas Sanitarias

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad

RF-05	Gestión de Auxiliares	Permitir el registro de catálogos dependientes (Categorías y Laboratorios) en la base de datos relacional para clasificación del catálogo maestro.	HU-06
RF-06	Alta de Medicamentos	Permitir la creación de medicamentos, garantizando mediante llaves primarias (PK) que el código de barras sea único en el sistema.	HU-07, CU-03
RF-07	Bandera de Controlados	Exigir la definición obligatoria del atributo booleano requiere_receta al registrar un medicamento, impactando el flujo de venta posterior.	HU-08
RF-08	Búsqueda Maestra	Ejecutar consultas tipo ILIKE en PostgreSQL para permitir la búsqueda de medicamentos ignorando mayúsculas y acentos.	HU-10

Módulo 3: Almacén y Restricciones Físicas

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad

RF-09	Recepción de Lotes	Permitir asociar múltiples cajas físicas a un mismo medicamento, registrando un Número de Lote y una Fecha de Caducidad obligatoria.	HU-12, CU-05
RF-10	Filtro de Caducidad	Bloquear a nivel de lógica de aplicación el registro de cualquier lote cuya fecha de caducidad sea igual o menor a la fecha actual.	HU-13
RF-11	Monitoreo de Riesgo	Generar consultas SQL que filtren y expongan únicamente los lotes cuya fecha de caducidad sea menor o igual a 90 días respecto a la fecha del servidor.	HU-15
RF-12	Deducción por Merma	Deducir el `stock_actual` de un lote dañado. La transacción debe ser rechazada por el constraint `CHECK (stock_actual >= 0)` si el retiro excede la existencia.	HU-16, CU-04

Módulo 4: Experiencia POS y Carrito

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad

RF-13	**Búsqueda Predictiva**	**Renderizar coincidencias de catálogo en la vista de cobro mediante intercepciones con patrón *Debounce* configurado a 300ms.**	**HU-17, CU-09**
RF-14	**Mutación de Carrito**	**Sumar cantidades en la misma fila (agrupar) si un producto idéntico es escaneado repetidamente, en lugar de crear nuevas líneas en el array del ticket temporal.**	**HU-18, CU-09**
RF-15	**Cálculo Financiero Realtime**	**Recalcular de manera reactiva y con precisión de coma flotante el Subtotal, IVA y Total a pagar tras cualquier mutación del carrito.**	**HU-21**

Módulo 5: Auditoría Médica Estricta

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad
RF-16	Condición de Receta	Detectar si en el carrito existe un ítem con requiere_receta = true y modificar el estado global del sistema a "Auditoría Médica Requerida".	HU-22, CU-10
RF-17	Bloqueo de Cobro	Inhabilitar por completo el evento de confirmación de venta y despliegue de pasarela de pago hasta que se capture una Cédula Profesional válida.	HU-24, CU-10

Módulo 6: Orquestación Distribuida (Saga)

ID	Nombre	Descripción Técnica (El sistema debe...)	Trazabilidad
RF-18	Validación de Pagos	Exigir que la suma matemática del dinero ingresado en pagos mixtos (Efectivo + Tarjeta) iguale el monto total del ticket antes de proceder a la orquestación.	HU-27
RF-19	Algoritmo FEFO	Deducir iterativamente en la base relacional las unidades requeridas, vaciando primero el lote activo	HU-40, CU-08

		con la fecha de caducidad más antigua.	
RF-20	Transaccionalidad Saga	Enviar petición de cobro a Node.js, quien descontará el stock en FastAPI. Si la generación del ticket en Firestore falla, Node.js debe emitir un comando HTTP para reintegrar el stock a FastAPI.	HU-28, HU-29, CU-11
RF-21	Resiliencia de Estado	Mantener intacto el array del carrito de compras en la memoria del cliente si el orquestador retorna un código HTTP 5xx o si ocurre una excepción de red.	HU-30

Módulo 7 y 8: Comprobantes, Anulaciones y Reportes

ID	*Nombre*	*Descripción Técnica (El sistema debe...)*	*Trazabilidad*
RF-22	*Generación Vectorial PDF*	*Construir nativamente un archivo PDF integrando el JSON de respuesta de venta, graficando debajo de cada producto los Lotes afectados.*	*HU-31, HU-32*

RF-23	*Reintegro Físico*	*Modificar el estado de un ticket a "Anulado" y enviar la orden de reintegro a FastAPI para devolver exactamente los lotes originales al inventario físico.*	*HU-37, CU-14*
RF-24	*Auditoría Financiera*	*Realizar funciones de agregación en Firestore para sumar el flujo de caja del día agrupado por método de pago y filtrado por usuario autenticado.*	*HU-35, HU-39*

7.3 Requerimientos No Funcionales (RNF) - ISO/IEC 25010

ID	Atributo ISO 25010	Descripción Técnica y Métrica
RNF-01	Eficiencia	Latencia: La búsqueda predictiva en el POS debe retornar resultados con una latencia < 500 ms (P95) implementando el patrón *Debounce*.
RNF-02	Eficiencia	Transaccionalidad: El flujo completo de orquestación de venta (Saga) debe resolverse en < 2000 ms entre los servidores.
RNF-03	Fiabilidad	Resiliencia Offline: Intercepción de `SocketException`. Ante fallos de red, el carrito no debe eliminarse, permitiendo reintento manual.

RNF-04	Seguridad	Comunicación B2B: Los endpoints críticos entre servidores (/descontar, /compensar) exigen inyección de la cabecera X-API-KEY.
RNF-05	Disponibilidad	Uptime: El sistema backend debe mantener una disponibilidad mínima del 99% durante el horario operativo comercial.

---

8. Product Backlog: Historias de Usuario y Casos de Uso

8.1 Listado de Historias de Usuario Priorizadas y Estimaciones (Scrum)

Cada Historia de Usuario (HU) ha sido redactada asegurando trazabilidad técnica entre el frontend (Flutter), la persistencia relacional (PostgreSQL) y los microservicios distribuidos (Node.js/Python). El Product Backlog se gestiona en Trello, categorizando la prioridad según el impacto en el negocio.

Reglas de Negocio Principales (Core Business Rules)

Antes de abordar las historias, todo el equipo debe respetar las siguientes reglas universales del dominio farmacéutico que rigen el sistema:

1. ***Integridad Física: Es matemáticamente imposible concretar una venta si el inventario físico es insuficiente. El sistema nunca debe permitir stock negativo.***
2. ***Despacho Sanitario (FEFO): Siempre se deben deducir primero las unidades de los lotes con la fecha de caducidad más próxima a vencer.***
3. ***Auditoría Estricta: Ningún medicamento con bandera de "Controlado/Antibiótico" puede ser vendido sin registrar la Cédula Profesional del médico responsable.***

4. ***Inmutabilidad Transaccional: Un ticket de venta generado no puede ser modificado ni borrado; cualquier corrección requiere una transacción de anulación formal y reintegro físico de los lotes.***

Definition of Done (DoD) Global

Para que el Product Owner apruebe cualquier HU de esta lista y la mueva a la columna "Terminado", debe cumplir obligatoriamente:

- ***[x] El código ha sido fusionado a la rama `main` en GitHub mediante Pull Request sin conflictos.***
- ***[x] Cumple al 100% con los Criterios de Aceptación definidos en la HU.***
- ***[x] El linter de Flutter (`flutter analyze`) o de Backend no arroja advertencias (Warnings).***
- ***[x] Se han manejado las excepciones de red (`try/catch`) para evitar cierres forzados del sistema.***

Módulo 1: Identidad y Perfiles (Auth & RBAC)

○

ID	Historia de Usuario	Criterios de Aceptación y CU	Responsables y Tareas Técnicas
HU-01 (5 SP) 🔴 Crítica	Inicio de Sesión Seguro: Como Cajero, quiero iniciar sesión con mi cuenta de Google para acceder al sistema de forma segura sin gestionar contraseñas.	1) Completarse en < 2 segundos. 2) Mostrar error si no hay permisos. CU-01	Daniel (3 SP): Integrar google_sign_in, UI de carga y enviar JWT. Samuel (2 SP): Endpoint POST /login, validar

			Firebase y emitir JWT.
HU-02 *(3 SP)* 🟡 *Media*	*Cierre de Sesión: Como Gerente, quiero cerrar mi sesión para evitar que otros empleados usen mis privilegios.*	*1) Cerrar sesión inmediatamente y redirigir a bienvenida.* *CU-01*	*Daniel (3 SP): Limpiar AuthBloc, borrar caché y destruir pila de navegación.*
HU-03 *(5 SP)* 🟠 *Alta*	*Restricción de Vistas Financieras: Como Gerente, quiero que los cajeros no tengan acceso a los reportes financieros ni a la gestión de usuarios.*	*1) Cajeros no deben ver ni navegar a módulos de administración.* *CU-02*	*Daniel (3 SP): Configurar Route Guards y renderizado leyendo JWT.* *Samuel (2 SP): Inyectar permisos en el payload JWT desde Firestore.*

HU-04 *(5 SP)* 🟡 *Media*	*Registro de Nuevos Empleados: Como Gerente, quiero registrar el correo de un empleado y asignarle su puesto (Cajero, Farmacéutico).*	*1) Solo usuarios administrativos pueden realizar la acción.* *CU-03*	*Daniel (3 SP): Formulario UI con roles y validación email.* *Samuel (2 SP): Endpoint POST /usuarios, guardar en Firestore.*
HU-05 *(3 SP)* 🟡 *Media*	*Revocación de Acceso a Empleados: Como Gerente, quiero desactivar la cuenta de un exempleado para que pierda acceso al sistema.*	*1) Impedir ingreso en su próximo intento de sesión.* *CU-03*	*Daniel (1 SP): Botón de desactivación en UI.* *Samuel (2 SP): Endpoint PATCH /usuarios/:id para baja lógica.*

Módulo 2: Catálogo de Medicamentos (Master Data)

○

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*

HU-06 *(5 SP)* 🟡 *Media*	*Gestión de Laboratorios y Categorías: Como Farmacéutico, quiero registrar a los proveedores y categorías terapéuticas.*	*1) Permitir guardar nombre y contacto del laboratorio.* *CU-04*	*Daniel (2 SP): Formularios de catálogos auxiliares.* *Karel (3 SP): Tablas PostgreSQL y Endpoints de creación.*
HU-07 *(5 SP)* 🟠 *Alta*	*Alta de Nuevos Medicamentos: Como Farmacéutico, quiero registrar un nuevo medicamento asociándolo a su laboratorio y categoría.*	*1) El código de barras debe ser único en todo el sistema.* *CU-04*	*Daniel (3 SP): Formulario responsivo con selectores.* *Karel (2 SP): Endpoint POST /medicamentos en FastAPI.*
HU-08 *(3 SP)*	*Bandera de Control Sanitario: Como Farmacéutico, quiero marcar si un medicamento es "Controlado/Antibiótico" al registrarlo.*	*1) Obligatorio definir si requiere receta antes de guardarlo.* *CU-04*	*Daniel (2 SP): Añadir Switch dinámico en el formulario.* *Karel (1 SP): Columna*

🔴 *Crítica*			*requiere_receta en DB.*
HU-09 *(3 SP)* 🟡 *Media*	*Actualización de Precios: Como Gerente, quiero actualizar el precio de venta de un medicamento.*	*1) El sistema no debe aceptar precios negativos o en cero.* *CU-04*	*Daniel (1 SP): Modal de edición rápida de precio.* *Karel (2 SP): Endpoint PUT /medicamentos/:id con Pydantic.*
HU-10 *(5 SP)* 🟠 *Alta*	*Búsqueda Maestra de Catálogo: Como Farmacéutico, quiero buscar un medicamento tecleando su nombre.*	*1) La búsqueda debe ignorar mayúsculas y acentos.* *CU-05*	*Daniel (3 SP): Buscador principal y tabla paginada.* *Karel (2 SP): Endpoint GET /medicamentos con ILIKE.*
HU-11 *(3 SP)*	*Retiro de Catálogo (Baja Lógica): Como Gerente, quiero marcar un medicamento como "Descontinuado".*	*1) Ocultarse de la venta, pero mantenerse en historial.*	*Daniel (1 SP): Opción de "Descontinuar" en UI.*

🟢 *Baja*		*CU-04*	*Karel (2 SP): Modificar activo = false vía PATCH.*

○

Módulo 3: Control de Almacén, Lotes y Caducidades

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*
HU-12 *(8 SP)* 🔴 *Crítica*	*Ingreso de Lotes: Como Almacenista, quiero registrar cajas físicas asociándolas a un Lote específico.*	*1) Permitir múltiples lotes diferentes del mismo medicamento.* *CU-06*	*Daniel (3 SP): UI de ingreso con captura de lote.* *Karel (5 SP): Tabla lotes_inventario y POST /lotes.*
HU-13	*Validación Estricta de Caducidad: Como Almacenista, quiero ingresar la fecha de*	*1) Impedir registro de lotes caducados o que caduquen hoy.*	*Daniel (3 SP): DatePicker restringido a fechas futuras.*

(5 SP) 🔴 *Crítica*	*caducidad obligatoria por caja.*	*CU-06*	*Karel (2 SP): Validación lógica en Python.*
HU-14 *(5 SP)* 🟠 *Alta*	*Consulta de Existencias: Como Gerente, quiero ver el desglose de lotes activos de un medicamento.*	*1) Mostrar lotes con stock > 0, ordenados por caducidad.* *CU-05*	*Daniel (3 SP): Listado anidado de detalles de lote.* *Karel (2 SP): Endpoint GET /medicamentos/:id/lotes.*
HU-15 *(8 SP)* 🟡 *Media*	*Monitoreo de Riesgo Sanitario: Como Gerente, quiero un panel con los lotes que caducarán en <= 90 días.*	*1) Mostrar listado enfocado en productos en riesgo.* *CU-07*	*Daniel (5 SP): Dashboard con alertas visuales de urgencia.* *Karel (3 SP): Query SQL filtrado (<= HOY + 90).*

HU-16 *(5 SP)* 🟠 *Alta*	*Declaración de Mermas: Como Almacenista, quiero retirar cajas dañadas o caducadas del stock físico.*	*1) Disminuir stock sin registrar ingresos económicos.* *CU-08*	*Daniel (2 SP): Formulario para registrar merma.* *Karel (3 SP): Endpoint POST /mermas para deducir stock.*

○

Módulo 4: Experiencia del Punto de Venta (Carrito)

○

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*
HU-17 *(8 SP)* 🔴 *Crítica*	*Búsqueda en Mostrador: Como Cajero, quiero buscar medicamentos velozmente para agilizar la fila.*	*1) Respuesta en < 500ms sin interrumpir escritura.* *CU-09*	*Daniel (8 SP): Autocomplete con Patrón Debounce (300ms) para optimizar red.*

HU-18 *(5 SP)* 🔴 *Crítica*	*Agregar al Ticket: Como Cajero, quiero agregar el medicamento al ticket temporal.*	*1) Si se agrega 2 veces, sumar cantidad en la misma fila.* *CU-10*	*Daniel (5 SP): Arquitectura de PosBloc y mutación inmutable.*
HU-19 *(3 SP)* 🟠 *Alta*	*Ajustar Cantidades: Como Cajero, quiero modificar cantidades directamente en el ticket.*	*1) Impedir cantidad < 1 (eliminar si es 0).* *CU-10*	*Daniel (3 SP): Controles de incremento/decremento en BLoC.*
HU-20 *(2 SP)* 🟡 *Media*	*Retirar Producto: Como Cajero, quiero retirar un producto si el cliente no lo lleva.*	*1) Opción visible y rápida para remover fila.* *CU-10*	*Daniel (2 SP): Evento de eliminación en BLoC.*

HU-21 *(5 SP)* 🔴 *Crítica*	*Cálculo Dinámico de Totales: Como Cajero, quiero ver subtotal, IVA y total en tiempo real.*	*1) Actualización instantánea al modificar productos.* *CU-10*	*Daniel (5 SP): Lógica matemática reactiva dentro del State.*

○

Módulo 5: Auditoría Médica (Reglas de Controlados)

○

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*
HU-22 *(5 SP)* 🟠 *Alta*	*Advertencia de Producto Controlado: Como Cajero, quiero que el sistema me advierta si agregué un antibiótico.*	*1) Mostrar alerta visual clara en la UI de cobro.* *CU-11*	*Daniel (5 SP): BlocBuilder que modifique el panel lateral.*

HU-23 *(5 SP)* 🔴 *Crítica*	*Captura de Datos de Receta: Como Cajero, quiero registrar Cédula y Nombre del Médico responsable.*	*1) Campos aparecen solo si hay controlados en ticket.* *CU-11*	*Daniel (5 SP): Campos condicionales y validaciones de longitud.*
HU-24 *(3 SP)* 🔴 *Crítica*	*Bloqueo de Venta Incompleta: Como Gerente, quiero que el sistema impida la venta si faltan datos de la receta.*	*1) Botón "Cobrar" inhabilitado hasta llenar la información.* *CU-11*	*Daniel (3 SP): Validación en UI para bloquear botón de confirmación.*

○

Módulo 6: Pagos y Orquestación de Venta

○

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*

HU-25 *(5 SP)* 🔴 *Crítica*	*Asistente de Cobro en Efectivo: Como Cajero, quiero calcular el cambio exacto.*	*1) Mostrar destacadamente el monto a devolver.* *CU-12*	*Daniel (5 SP): Ventana modal interactiva con calculadora.*
HU-26 *(2 SP)* 🟡 *Media*	*Cobro con Tarjeta: Como Cajero, quiero indicar el pago por terminal.*	*1) Registrar método en la transacción final.* *CU-12*	*Daniel (1 SP): Selección de tarjeta.* *Samuel (1 SP): Soporte en BD.*
HU-27 *(3 SP)* 🟡 *Media*	*Pago Mixto: Como Cajero, quiero dividir cuenta (efectivo/tarjeta).*	*1) Suma de montos debe igualar total exacto.* *CU-12*	*Daniel (2 SP): Validación matemática.* *Samuel (1 SP): Arreglo de pagos.*

HU-28 *(8 SP)* 🔴 *Crítica*	*Confirmación de Venta: Como Cajero, quiero procesar la venta y generar folio.*	*1) Registro permanente. 2) Vaciar carrito.* *CU-13*	*Samuel (5 SP): Endpoint receptor y guardado en Firestore.* *Daniel (3 SP): Petición HTTP y limpiar BLoC.*
HU-29 *(8 SP)* 🔴 *Crítica*	*Deducción Automática de Inventario: Como Gerente, quiero que al vender se reduzca el inventario.*	*1) Venta no se concreta si falla la reducción.* *CU-13*	*Samuel (3 SP): Orquestar Patrón Saga.* *Karel (5 SP): Endpoint receptor en FastAPI.*
HU-30 *(5 SP)*	*Prevención por Caída de Red: Como Cajero, quiero saber si falló la red sin perder mi carrito.*	*1) Alerta de error. 2) Mantener carrito intacto.* *CU-14*	*Daniel (5 SP): Interceptar SocketException en Flutter.*

🟠 *Alta*			

-

Módulo 7: Generación de Comprobantes (Tickets PDF)

-

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*
HU-31 *(13 SP)* 🟠 *Alta*	*Emisión de Ticket Comercial: Como Cliente, quiero un comprobante detallado con mis productos.*	*1) Generar documento digital estandarizado (80mm).* *CU-15*	*Daniel (13 SP): Motor de renderizado pdf nativo en Flutter.*
HU-32 *(5 SP)* 🟠 *Alta*	*Trazabilidad de Lotes: Como Cliente, quiero que mi ticket especifique el Lote de mi medicamento.*	*1) Mostrar lote debajo del producto en el recibo.* *CU-15*	*Daniel (5 SP): Mapear respuesta BD e integrarlo al PDF.*

HU-33 *(3 SP)* 🟡 *Media*	*Constancia de Receta: Como Cliente, quiero que mi recibo indique datos de mi médico si aplica.*	*1) Mostrar info médica al pie de página condicionalmente.* *CU-15*	*Daniel (3 SP): Evaluar datos médicos y renderizar bloque.*
HU-34 *(5 SP)* 🟡 *Media*	*Previsualización de Comprobante: Como Cajero, quiero visualizar el ticket para imprimirlo.*	*1) Mostrar visor nativo tras cobrar.* *CU-15*	*Daniel (5 SP): Implementar paquete printing en Flutter.*

○

Módulo 8: Arquitectura Transaccional y Reportes Administrativos

○

ID	*Historia de Usuario*	*Criterios de Aceptación y CU*	*Responsables y Tareas Técnicas*

HU-35 *(5 SP)* 🟡 *Media*	*Consulta de Turno: Como Cajero, quiero revisar mis ventas del día para el corte.*	*1) Ver solo tickets procesados por el mismo usuario.* *CU-16*	*Daniel (2 SP): Interfaz.* *Samuel (3 SP): Endpoint filtrado por UID.*
HU-36 *(5 SP)* 🟡 *Media*	*Auditoría Global: Como Gerente, quiero acceder al historial completo de ventas.*	*1) Historial sin restricción de cajero.* *CU-17*	*Daniel (2 SP): Tabla paginada.* *Samuel (3 SP): Endpoint global.*
HU-37 *(8 SP)* 🟠 *Alta*	*Anulación de Operaciones: Como Gerente, quiero cancelar un ticket y devolver mercancía.*	*1) Ticket se marca "Anulado" y stock se restablece.* *CU-18*	*Daniel (2 SP): UI anulación.* *Samuel (3 SP): Saga Rollback.*

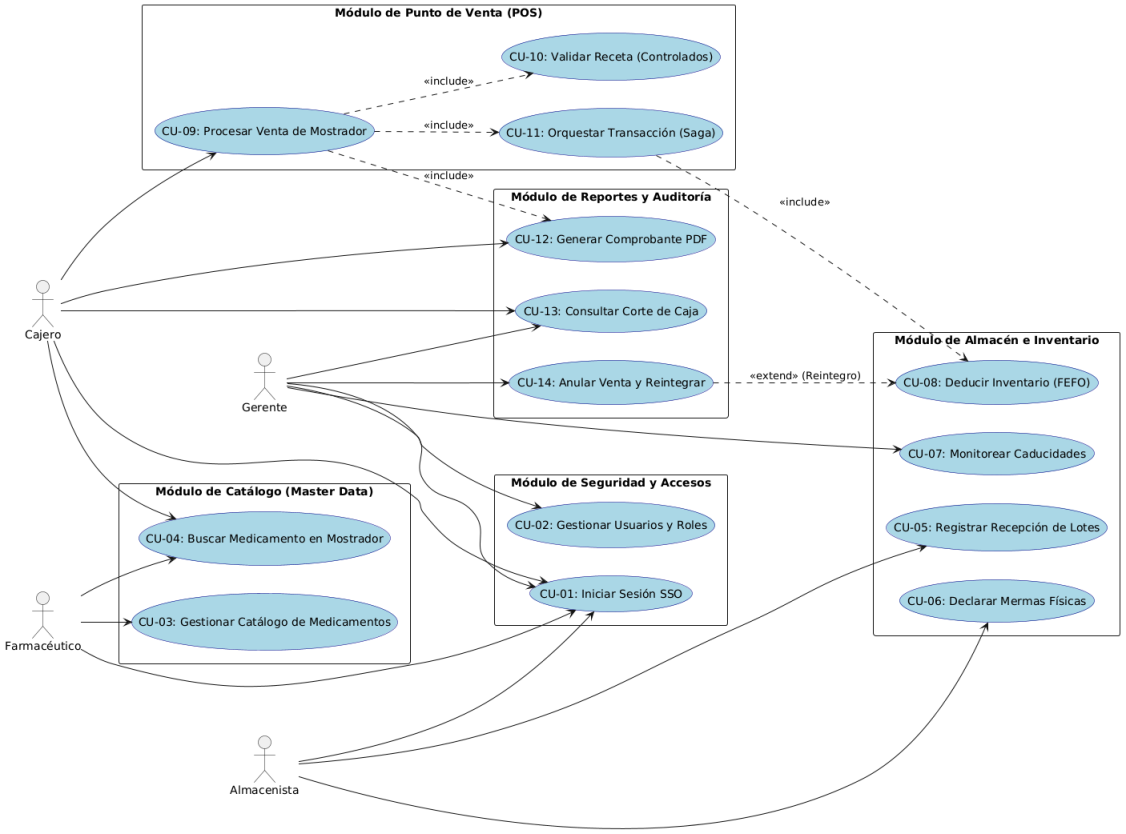
			Karel (3 SP): Reintegro SQL.
HU-38 *(5 SP)* 🟡 *Media*	*Reporte Sanitario: Como Farmacéutico, quiero generar un reporte con recetas retenidas.*	*1) Extraer listado exclusivo de auditorías médicas.* *CU-19*	*Daniel (2 SP): Pantalla reporte.* *Samuel (3 SP): Consulta Firestore.*
HU-39 *(5 SP)* 🟡 *Media*	*Resumen Financiero: Como Gerente, quiero un resumen de ingresos (efectivo/tarjetas).*	*1) Métricas consolidadas del flujo de caja diario.* *CU-16*	*Daniel (2 SP): Tarjetas KPI.* *Samuel (3 SP): Agregación backend.*
HU-40 *(13 SP)*	*Despacho Inteligente (FEFO): Como Gerente, quiero descontar las cajas más viejas primero sin sobreventas.*	*1) Impedir ventas sin stock físico.*	*Karel (13 SP): Algoritmo FEFO, Pessimistic Locking y Transacciones ACID.*

🔴 ***Crítica***		***2) Priorizar caducidad.*** ***CU-20***	

○

8.2 Diagrama General de Casos de Uso (UML)

El siguiente diagrama modela los límites funcionales del sistema, organizado por módulos de negocio. Ilustra la interacción entre los actores físicos (Cajero, Gerente, Farmacéutico, Almacenista) y las transacciones de alto nivel, incluyendo los procesos internos del sistema como la validación sanitaria y la deducción inteligente de inventario.



8.3 Especificación Formal de Casos de Uso (Core Business)

(Nota: Para mantener la concisión requerida en la documentación de Arquitectura Orientada a Servicios, se documentan rigurosamente los casos de uso transaccionales y de mayor complejidad, dejando implícitos los flujos CRUD básicos).

CU-01: Iniciar Sesión SSO

- ***Actor Primario: Empleado (Cajero, Gerente, Farmacéutico, Almacenista).***
- ***Actores Secundarios: Orquestador (Node.js), Firebase Auth.***
- ***Precondiciones: El empleado posee una cuenta de Google vinculada al sistema.***
- ***Flujo Principal:***
    1. ***El empleado selecciona "Ingresar con Google" en el cliente Flutter.***
    2. ***Flutter obtiene el `idToken` y lo envía a Node.js.***
    3. ***Node.js verifica la firma criptográfica, extrae el rol desde Firestore y emite un JWT firmado.***
- ***Postcondiciones: El estado local muta a "Autenticado" y la UI habilita las rutas según el rol (RBAC).***

CU-03: Gestionar Catálogo de Medicamentos

- ***Actor Primario: Farmacéutico.***
- ***Actores Secundarios: Servicio de Inventario (FastAPI), PostgreSQL.***
- ***Precondiciones: JWT con permisos administrativos.***
- ***Flujo Principal:***
    1. ***El Farmacéutico captura los datos del medicamento, incluyendo la bandera booleana `requiere_receta`.***
    2. ***Flutter envía el DTO al backend FastAPI.***
    3. ***FastAPI valida el payload con `Pydantic` e inserta el registro en PostgreSQL.***
- ***Postcondiciones: El medicamento queda disponible para recepción de lotes y búsqueda en mostrador.***

CU-05: Registrar Recepción de Lotes

- ***Actor Primario: Almacenista.***
- ***Actores Secundarios: Servicio de Inventario (FastAPI).***
- ***Precondiciones: El medicamento existe en el catálogo maestro.***
- ***Flujo Principal:***

    1. *El Almacenista captura el Código de Barras, Lote y Fecha de Caducidad.*
    2. *El cliente valida que la fecha sea futura.*
    3. *FastAPI guarda el registro en `lotes_inventario`.*
- *Postcondiciones: El lote queda habilitado inmediatamente para el despacho FEFO.*
- *Flujos Alternativos: Si la fecha es inválida o el formato del lote es incorrecto, el sistema rechaza la operación sin tocar la base de datos.*

CU-09: Procesar Venta de Mostrador

- *Actor Primario: Cajero.*
- *Actores Secundarios: Cliente (Físico).*
- *Precondiciones: El cajero ha buscado productos y los ha añadido al carrito de compras reactivo (BLoC).*
- *Flujo Principal:*
    1. *El Cajero presiona el botón "Cobrar".*
    2. *El sistema invoca automáticamente el CU-10 (Validar Receta) si existen medicamentos controlados en el carrito.*
    3. *El Cajero captura el método de pago (Efectivo/Tarjeta).*
    4. *El sistema invoca el CU-11 (Orquestar Transacción) para procesar el pago.*
    5. *El sistema invoca el CU-12 (Generar Comprobante PDF) y limpia la pantalla.*
- *Postcondiciones: La venta se concreta y el sistema queda listo para el siguiente cliente.*
- *Flujos Alternativos (Resiliencia): Si durante el proceso el cliente Flutter detecta una `SocketException` (caída de red), el BLoC atrapa el error, no borra el carrito y muestra un botón de "Reintento Manual".*

CU-10: Validar Receta (Controlados)

- *Actor Primario: Cajero (Invocado por el Sistema).*
- *Precondiciones: El carrito contiene un producto con `requiere_receta = true`.*
- *Flujo Principal:*
    1. *La interfaz gráfica se bloquea preventivamente en rojo.*
    2. *El Cajero transcribe la Cédula Profesional y Nombre del Médico desde la receta física.*
    3. *Flutter valida la longitud de los campos.*
- *Postcondiciones: El bloqueo se levanta y los datos médicos se anexan al DTO de la venta.*

CU-11: Orquestar Transacción (Saga)

- ***Actor Primario: Orquestador (Node.js).***
- ***Actores Secundarios: Servicio de Inventario (FastAPI), Firestore.***
- ***Precondiciones: Petición de venta recibida desde Flutter.***
- ***Flujo Principal:***
    1. ***Node.js recibe la petición y llama de forma síncrona a FastAPI para invocar el CU-08 (Deducir Inventario FEFO).***
    2. ***FastAPI confirma el descuento exitoso de las cajas físicas.***
    3. ***Node.js genera un Folio Único y guarda el ticket inmutable en Firestore.***
    4. ***Node.js responde `200 OK` al cliente.***
- ***Postcondiciones: Ticket histórico generado y base de datos relacional actualizada.***
- ***Flujos Alternativos (Compensación): Si en el paso 3 Firestore sufre una caída, Node.js atrapa el error, dispara un llamado de compensación a FastAPI para reintegrar el stock retenido, y devuelve `500 Internal Error` al cliente.***

CU-08: Deducir Inventario (FEFO)

- ***Actor Primario: Servicio de Inventario (FastAPI).***
- ***Actores Secundarios: PostgreSQL.***
- ***Precondiciones: Petición de deducción recibida desde el Orquestador.***
- ***Flujo Principal:***
    1. ***FastAPI inicia transacción SQL y ejecuta un `SELECT ... FOR UPDATE` sobre los lotes del medicamento.***
    2. ***Ordena los lotes válidos mediante `ORDER BY fecha_caducidad ASC`.***
    3. ***Deduce iterativamente unidades del lote más próximo a vencer.***
    4. ***Hace `commit` en la base de datos y retorna los lotes modificados.***
- ***Postcondiciones: Se asegura la rotación sanitaria correcta del producto.***
- ***Flujos Alternativos: Si la suma de todos los lotes es insuficiente, el sistema ejecuta un `rollback`, retornando error por falta de stock. El constraint `CHECK (stock_actual >= 0)` en PostgreSQL actúa como última barrera defensiva.***

CU-14: Anular Venta y Reintegrar

- ***Actor Primario: Gerente.***
- ***Actores Secundarios: Node.js, FastAPI.***
- ***Precondiciones: Ticket emitido existente en el histórico.***
- ***Flujo Principal:***
    - ***El Gerente anula el folio en el sistema.***
    - ***Node.js marca el ticket como "Anulado" en Firestore y extrae los lotes originales.***
    - ***Node.js envía la orden de reintegro a FastAPI.***
    - ***FastAPI suma el stock devuelto a los lotes originales en PostgreSQL.***
- ***Postcondiciones: Inventario físico reintegrado y ticket invalidado financieramente.***

- **[BLOQUE EN REVISIÓN Y DESARROLLO EXHAUSTIVO]**

---

9. Estrategia Integral de Pruebas (Estándar IEEE 829)

9.1 Plan de Pruebas (Test Plan)

- **Alcance: Validación de la UI reactiva, orquestación REST y restricciones (Constraints) de BD.**
- **Entorno: Entorno local aislando los servicios y entorno *Staging* en la nube.**
- **Estrategia: Pruebas unitarias sobre el estado (BLoC), pruebas de integración sobre el patrón Saga, y simulaciones End-to-End (E2E) para caídas de red. Las pruebas unitarias del cliente pueden ejecutarse mediante el framework de *testing* integrado en Flutter, permitiendo validar automáticamente el comportamiento de los componentes de negocio independientes de la UI.**

9.2 Diseño y Especificación de Pruebas

9.2.1 Documentación de Pruebas del Cliente (Flutter)

- **CP-CLI-01: Búsqueda Predictiva con Debounce (Rendimiento)**
    - ***Acción:* Teclear "Amoxicilina" en intervalos de 50ms por carácter.**
    - ***Aserción:* El cliente no dispara peticiones HTTP por cada letra tecleada, sino una sola al finalizar (300ms después de la última tecla), optimizando el ancho de banda y cumpliendo el RNF-01.**

- **CP-CLI-02: Bloqueo de UI por Medicamento Controlado (Lógica BLoC)**
  - ***Acción:*** **Inyectar Medicamento(requiere_receta: true) al carrito.**
  - ***Aserción:*** **La interfaz renderiza dinámicamente los campos de "Cédula Médica" y el botón de "Cobrar" queda en estado onPressed: null (inhabilitado) hasta que la cédula supere la validación de longitud.**
- **CP-CLI-03: Resiliencia ante Desconexión (Tolerancia a Fallos - E2E)**
  - ***Acción:*** **Interrumpir la conexión a internet (Modo avión) y procesar la venta.**
  - ***Aserción:*** **La app atrapa la SocketException, muestra un SnackBar de error indicando fallo de red y no borra el carrito de compras, habilitando el botón de reintento manual sin sufrir cierres inesperados.**

9.2.2 Documentación de Pruebas del Servidor

[ÁREA PARA SAMUEL Y KAREL: Insertar CP-SRV-01 (Validación de Token JWT), CP-SRV-02 (Validación de Constraints FEFO en SQL) y CP-SRV-03 (Compensación Saga en Node.js)]

9.3 Reporte de Incidentes y Resumen de Ejecución

(Sección reservada para documentar los incidentes detectados durante la fase de validación e integración final).

---

10. Métricas de Control Ágil y Definition of Done (DoD)

10.1 Métricas de Rendimiento y Control Ágil

Para monitorear el progreso y la salud del código, se establecen las siguientes métricas:

1. **Gráfica de Quemado (Burndown Chart): Se utiliza esta herramienta visual al final de cada *Sprint* para comparar la cantidad de *Story Points* planificados contra los completados, permitiendo medir la velocidad real del equipo (*Velocity*) e identificar cuellos de botella en la integración. *(Nota: Insertar imagen de gráfica de quemado exportada de Trello)*.**
2. **SLAs de Latencia: Las peticiones de lectura deben resolver en < 500 ms. Las transacciones de orquestación en < 2000 ms.**

3. Control de Deuda Técnica: Ejecución obligatoria de flutter analyze exigiendo cero advertencias (*Warnings*) antes de realizar el *Merge* a la rama principal.

10.2 Criterios de Aceptación y DoD Exhaustivo

Para que una Historia de Usuario o incremento sea "Terminado" (Done), debe cumplir rigurosamente:

- [x] El código fue integrado a main mediante *Pull Request* aprobado, sin conflictos en GitHub.
- [x] Cumple al 100% con los Criterios de Aceptación especificados en el *Product Backlog*.
- [x] La lógica de red en el cliente incluye manejo de excepciones, garantizando degradación elegante sin cierres forzados (*Crashes*).
- [x] Las rutas de las APIs y repositorios cuentan con documentación técnica.
- [x] La interfaz gráfica ha sido probada sin presentar desbordamientos de diseño (*Overflow*).

---

11. Despliegue, Reportes y Demostración Funcional

11.1 Arquitectura de Despliegue y Flujo de CI/CD

Para cumplir con los requerimientos de disponibilidad, el ecosistema se encuentra publicado en plataformas separadas en la nube. El flujo de despliegue se realiza mediante Integración Continua (CI/CD) utilizando el repositorio GitHub como fuente principal del código. Cada actualización aprobada en la rama principal desencadena el despliegue automático de los servicios:

1. Frontend (Flutter Web): Alojado en Firebase Hosting, garantizando la entrega rápida de los estáticos del cliente a través de su CDN global.
2. Backend (Node.js & FastAPI): Desplegados de forma automática como servicios web independientes en Render (Platform as a Service), conectándose de forma segura a las bases de datos en la nube (Neon Tech / Supabase para PostgreSQL y Google Cloud para Firestore).

11.2 Módulo de Generación de Reportes del Sistema

El sistema cumple con las normativas de auditoría mediante la generación de reportes digitales. Para delegar la carga de procesamiento

al cliente y simplificar la arquitectura, la generación del archivo PDF se realiza nativamente en Flutter. Utilizando el paquete pdf, el cliente procesa la respuesta JSON de la venta exitosa y dibuja mediante código (Canvas) un comprobante térmico de 80mm que incluye el Folio, Lotes afectados y Cédula Médica.

11.3 Evidencia de Funcionalidad: Videos Explicativos

- **Video Demostrativo del Cliente (Frontend):**
- **Video Demostrativo del Servidor (APIs/BD):**

---

12. Autoevaluación del Equipo de Desarrollo

---

13. Enlaces y Repositorios Oficiales del Proyecto

- **Repositorio Monorepo GitHub:** https://github.com/ADanicode/Farmacia-POS-AWOS
- **Tablero Scrum Trello:** https://trello.com/invite/b/69a86fe063091e623402bfe2/ATTIfe1659d9a7db786135ca03609bd22aca645C7B91/farmacia
- **Drive:** https://drive.google.com/drive/folders/1gysIV7s-jfFX_GMZi1Way-62bSYimr79?usp=drive_link
- **Whatsapp:**

  https://chat.whatsapp.com/ChiOxt7S5o0FaW0KpD9R3x

---

14. Bibliografía y Referencias Técnicas

1. Felangel. (2024). *Bloc Library Documentation*. Recuperado de: https://bloclibrary.dev/
2. PostgreSQL Global Development Group. (2024). *PostgreSQL 18 Documentation (Concurrency and Row-level Locking)*. Recuperado de: https://www.postgresql.org/docs/

3. Tiangolo. (2024). *FastAPI Documentation*. Recuperado de: https://fastapi.tiangolo.com/
4. Google. (2024). *Firebase Admin SDK and Firestore Docs*. Recuperado de: https://firebase.google.com/docs
5. IEEE Standards Association. *IEEE 830-1998 (Software Requirements Specification)* e *IEEE 829-2008 (Test Documentation)*.